

# Week 5

## Problem 1-The Maximum Subarray

```
import java.io.*;

import java.math.*;

import java.security.*;

import java.text.*;

import java.util.*;

import java.util.concurrent.*;

import java.util.function.*;

import java.util.regex.*;

import java.util.stream.*;

import static java.util.stream.Collectors.joining;

import static java.util.stream.Collectors.toList;

class Result {

    public static List<Integer> maxSubarray(List<Integer> arr) {

        int maxSubarray = arr.get(0);

        int currentSum = arr.get(0);

        int maxSubsequence = 0;

        int maxElement = arr.get(0);

        for (int i = 0; i < arr.size(); i++) {

            int num = arr.get(i);

            if (num > 0) {

                maxSubsequence += num;

            }

            maxElement = Math.max(maxElement, num);

            if (i > 0) {

                currentSum = Math.max(num, currentSum + num);

                maxSubarray = Math.max(maxSubarray, currentSum);

            }

        }

    }

}
```

```

    }

    if (maxSubsequence == 0) {
        maxSubsequence = maxElement;
    }

    return Arrays.asList(maxSubarray, maxSubsequence);
}

}

public class Solution {

    public static void main(String[] args) throws IOException {

        BufferedReader bufferedReader = new BufferedReader(new
InputStreamReader(System.in));

        BufferedWriter bufferedWriter = new BufferedWriter(new
FileWriter(System.getenv("OUTPUT_PATH")));

        int t = Integer.parseInt(bufferedReader.readLine().trim());

        IntStream.range(0, t).forEach(tltr -> {

            try {

                int n = Integer.parseInt(bufferedReader.readLine().trim());

                List<Integer> arr = Stream.of(bufferedReader.readLine().replaceAll("\\s+$", "").split(" "))

                    .map(Integer::parseInt)

                    .collect(toList());

                List<Integer> result = Result.maxSubarray(arr);

                bufferedWriter.write(

                    result.stream()

                        .map(Object::toString)

                        .collect(joining(" "))

                        + "\n"

                );

            } catch (IOException ex) {

                throw new RuntimeException(ex);

            }

        });

        bufferedReader.close();

```

```
        bufferedWriter.close();
    }
}
```

Output

## Problem 2-Subarray Division

```
import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;

class Result {

    public static int birthday(List<Integer> s, int d, int m) {
        int count=0;
        int sum=0;
        for(int i=0;i<m;i++){
            sum+=s.get(i);
        }
        if(sum==d){
            count++;
        }
        for(int i=m;i<s.size();i++){
            sum+=s.get(i);
            sum-=s.get(i-m);
            if(sum==d){
```

```

        count++;
    }
}

return count;
}
}

public class Solution {

    public static void main(String[] args) throws IOException {

        BufferedReader bufferedReader = new BufferedReader(new
InputStreamReader(System.in));

        BufferedWriter bufferedWriter = new BufferedWriter(new
FileWriter(System.getenv("OUTPUT_PATH")));

        int n = Integer.parseInt(bufferedReader.readLine().trim());

        List<Integer> s = Stream.of(bufferedReader.readLine().replaceAll("\\s+$", "").split(" "))
            .map(Integer::parseInt)
            .collect(toList());

        String[] firstMultipleInput = bufferedReader.readLine().replaceAll("\\s+$", "").split(" ");

        int d = Integer.parseInt(firstMultipleInput[0]);

        int m = Integer.parseInt(firstMultipleInput[1]);

        int result = Result.birthday(s, d, m);

        bufferedWriter.write(String.valueOf(result));

        bufferedWriter.newLine();

        bufferedReader.close();

        bufferedWriter.close();

    }
}

```

## Problem 3- String to Integer (atoi)

```

class Solution {

    public int myAtoi(String s) {

        int i = 0;

        int n = s.length();
    }
}

```

```

while (i < n && s.charAt(i) == ' ') {
    i++;
}
int sign = 1;
if (i < n && s.charAt(i) == '-') {
    sign = -1;
    i++;
} else if (i < n && s.charAt(i) == '+') {
    i++;
}
int result = 0;
while (i < n && Character.isDigit(s.charAt(i))) {
    int digit = s.charAt(i) - '0';

    if (result > (Integer.MAX_VALUE - digit) / 10) {
        if (sign == 1) {
            return Integer.MAX_VALUE;
        } else {
            return Integer.MIN_VALUE;
        }
    }

    result = result * 10 + digit;
    i++;
}
return result * sign;
}
}

```

## Problem 4- Naive algorithm for Pattern Searching

```

import java.util.*;

class GFG {

```

```

static ArrayList<Integer> search(String pat, String txt) {
    int m = pat.length();
    int n = txt.length();
    ArrayList<Integer> ans = new ArrayList<>();
    for (int i = 0; i <= n - m; i++) {
        int j;
        for (j = 0; j < m; j++) {
            if (txt.charAt(i + j) != pat.charAt(j)) {
                break;
            }
        }
        if (j == m) {
            ans.add(i);
        }
    }
    return ans;
}

public static void main(String[] args) {
    String txt = "aabaacaadaabaaba";
    String pat = "aaba";
    ArrayList<Integer> result = search(pat, txt);
    for (int index : result) {
        System.out.print(index + " ");
    }
}

```

## Problem 5- Maximum Subarray

```

class Solution {
    public int maxSubArray(int[] nums) {
        int currentSum = nums[0];

```

```

int maxSum = nums[0];

for (int i = 1; i < nums.length; i++) {

    currentSum = Math.max(nums[i], currentSum + nums[i]);

    maxSum = Math.max(maxSum, currentSum);

}

return maxSum;

}

}

```

## Problem 6- Longest Substring Without Repeating Characters

```

class Solution {

    public int lengthOfLongestSubstring(String s) {

        HashSet<Character> set = new HashSet<>();

        int left = 0;

        int maxLength = 0;

        for (int right = 0; right < s.length(); right++) {

            while (set.contains(s.charAt(right))) {

                set.remove(s.charAt(left));

                left++;

            }

            set.add(s.charAt(right));

            maxLength = Math.max(maxLength, right - left + 1);

        }

        return maxLength;

    }

}

```

## Problem 7- -Find and Replace Pattern

```

class Solution {

    public List<String> findAndReplacePattern(String[] words, String pattern) {

        List<String> result = new ArrayList<>();

    }

}

```

```

    for (String word : words) {
        if (matches(word, pattern)) {
            result.add(word);
        }
    }
    return result;
}

public boolean matches(String word, String pattern) {
    HashMap<Character, Character> map = new HashMap<>();
    HashSet<Character> used = new HashSet<>();
    for (int i = 0; i < word.length(); i++) {
        char p = pattern.charAt(i);
        char w = word.charAt(i);
        if (map.containsKey(p)) {
            if (map.get(p) != w) {
                return false;
            }
        }
        else {
            if (used.contains(w)) {
                return false;
            }
            map.put(p, w);
            used.add(w);
        }
    }
    return true;
}
}

```

## Problem 8- Alternating Characters

```
import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;

class Result {

    public static int alternatingCharacters(String s) {
        int deletions = 0;
        for (int i = 1; i < s.length(); i++) {
            if (s.charAt(i) == s.charAt(i - 1)) {
                deletions++;
            }
        }
        return deletions;
    }
}

public class Solution {

    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new
        InputStreamReader(System.in));

        BufferedWriter bufferedWriter = new BufferedWriter(new
        FileWriter(System.getenv("OUTPUT_PATH")));

        int q = Integer.parseInt(bufferedReader.readLine().trim());
```

```

IntStream.range(0, q).forEach(qltr -> {
    try {
        String s = bufferedReader.readLine();
        int result = Result.alternatingCharacters(s);
        bufferedWriter.write(String.valueOf(result));
        bufferedWriter.newLine();
    } catch (IOException ex) {
        throw new RuntimeException(ex);
    }
});
bufferedReader.close();
bufferedWriter.close();
}
}

```

## Problem 9- String Matching in an Array

```

class Solution {
    public List<String> stringMatching(String[] words) {
        List<String> result = new ArrayList<>();
        for (int i = 0; i < words.length; i++) {
            for (int j = 0; j < words.length; j++) {
                if (i != j && words[j].contains(words[i])) {
                    result.add(words[i]);
                    break;
                }
            }
        }
        return result;
    }
}

```

## Problem 10- Maximum Sum Circular Subarray

```
class Solution {  
    public int maxSubarraySumCircular(int[] nums) {  
        int totalSum=0;  
        int currentMax=0;  
        int maxSum=nums[0];  
        int currentMin=0;  
        int minSum=nums[0];  
        for(int num:nums){  
            currentMax =Math.max(num,currentMax+num);  
            maxSum=Math.max(maxSum,currentMax);  
            currentMin=Math.min(num,currentMin+num);  
            minSum=Math.min(minSum,currentMin);  
            totalSum+=num;  
        }  
        if(maxSum<0){  
            return maxSum;  
        }  
        return Math.max(maxSum,totalSum-minSum);  
    }  
}
```